

Contexto y rol del estudiante

[Ir a la presentación completa del Proyecto Integrador](#)

Gestor estratégico de tareas

Contexto del proyecto y rol del estudiante:

MateCode es una startup que desarrolla aplicaciones web para pequeñas empresas. En este caso, un cliente necesita una aplicación web para que sus empleados puedan gestionar tareas diarias de forma organizada, persistente y accesible desde cualquier dispositivo.

La empresa solicita una solución rápida, escalable y basada en servicios administrados (BaaS), por lo que se define el siguiente stack:

- Frontend con React + TypeScript
- Backend as a Service con Firebase (Auth + Firestore)
- Notificaciones por email con AWS SES
- Deploy en Vercel
- Tests de componentes con Vitest y React Testing Library.

Como Desarrollador/a Full Stack Junior, tu rol será diseñar, implementar, testear y documentar una aplicación SPA funcional que permita:

- Registro y autenticación de usuarios
- Gestión completa de tareas (CRUD)
- Persistencia en la nube por usuario
- Envío de notificaciones por email
- Deploy público accesible vía URL

Deberás aplicar buenas prácticas de arquitectura, tipado, testing, seguridad y control de versiones profesional de código.

Objetivos del Proyecto Integrador

 Objetivos

- Utilizar herramientas de AI como apoyo al desarrollo, redactando prompts claros y documentando su uso, priorizando siempre la comprensión del código.
- Construir interfaces modulares utilizando JSX, componentes reutilizables, props y manejo de estado para organizar y comunicar datos entre componentes.
- Aplicar TypeScript para definir estructuras de datos claras y reducir errores durante el desarrollo.
- Diseñar interfaces responsive con enfoque mobile first e implementar navegación en aplicaciones SPA.
- Integrar formularios, autenticación de usuarios y operaciones CRUD con servicios como Firebase y Firestore.
- Buenas prácticas de desarrollo: Aplicar testing, control de versiones con Git y despliegue de aplicaciones en entornos de producción.

Consigna

Detalle

Construir una aplicación web SPA de gestión de tareas con autenticación de usuarios, persistencia en la nube, envío de notificaciones por email y deploy en producción, aplicando buenas prácticas de arquitectura, tipado con TypeScript, testing y control de versiones.

Alcance funcional mínimo

Autenticación:

- Registro con email y password y/o con Google.
- Login y logout.
- Protección de rutas privadas (no se debe permitir ver las tareas del usuario antes de que se registre y loguee).
- Manejo claro de errores de autenticación.

Gestión de tareas (por usuario autenticado):

- Crear tarea (título + descripción).
- Listar tareas del usuario.
- Editar tarea.
- Eliminar tarea.
- Marcar como completada.

- Persistencia en Firestore.
- Cada usuario solo puede ver sus propias tareas.

Persistencia y sincronización

- Datos almacenados en Cloud Firestore.
- Filtrado por userId.
- Manejo de estados de carga y errores.
- Actualización automática de la UI tras operaciones CRUD.

Email

- Permitir mandar un email con un resumen del estado de todas las tareas mediante un botón.
- La función debe usar AWS SES para enviar un email de confirmación.
- No se deben exponer secretos en el frontend.

Testing

- Tests unitarios de funciones clave.
- Tests de componentes principales (ej: TodoForm, TodoList).
- Mock de servicios externos cuando corresponda.

Deploy

- Deploy en Vercel.
- URL pública funcional.
- Variables de entorno configuradas correctamente.

Seguridad

- Crear .env para desarrollo y .env.example sin data
- Las credenciales de AWS deben manejarse como variables de entorno.
- Las credenciales de Firebase deben manejarse como variables de entorno.
- Añadir .env en el archivo .gitignore.
- Commits realizados excluyendo variables de entorno.

Extra credit

1. Agregar filtros: mostrar tareas completadas, pendientes o todas.
2. Implementar drag & drop para reordenar tareas, puedes explorar librerías como dnd-kit.
3. Agregar fechas de vencimiento y ordenar por prioridad.

Estos extras no son obligatorios, pero pueden ayudarte a demostrar iniciativa y profundidad técnica.

Entregable Final

Entregable

Desarrollo

 Código organizado por capas (components, pages, services, hooks, types, api)

 Commits semánticos y descriptivos

 Aplicación desplegada en Vercel con:

- Autenticación funcional.
- CRUD completo persistente en Firestore.
- Envío real de email mediante AWS SES.
- AWS SES invocado a través de Vercel Functions.

Documentación

 README con:

- Descripción del proyecto.
- Decisiones arquitectónicas.
- Instrucciones de instalación.
- Variables de entorno necesarias.
- URL de producción.
- Flujo de envío de emails.
- Explica cómo integraste la IA en tu proceso de trabajo, en qué situaciones fue más efectiva y qué patrones o buenas prácticas descubriste a partir de su uso.

 Entregable final: Enlace con el repositorio de GitHub

Estructura sugerida del proyecto (simple):

project-root/

		src/	
		pages/	# Vistas (Login, Register, Tasks)
		components/	# Componentes UI (TodoForm, TodoList, etc.)
		features/	# Lógica por dominio (auth, tasks)
		services/	# Integraciones (Firebase/Auth/Firestore, API calls)
		routes/	# Router + rutas privadas (ProtectedRoute)
		hooks/	# Hooks (useAuth, useTasks)
		types/	# Tipos e interfaces compartidas
		utils/	# Helpers (validaciones, formateo, etc.)
		functions/	# Vercel Functions (API / envío de emails con SES)
		tests/	# Unit tests + tests de componentes + mocks
		.env	# Local (NO se sube)
		.env.example	# Plantilla sin secretos (SÍ se sube)
		.gitignore	# Debe excluir .env
		README.md	# Setup, scripts, variables de entorno, deploy

Rúbrica de corrección

En [este archivo](#) podrás ver las rúbricas de corrección para el Proyecto Integrador del Módulo.

Guía para el desarrollo del PI

[Acceder a la guía para de acompañamiento para el desarrollo del PI 4](#)

GUÍA PARA EL DESARROLLO DEL PROYECTO INTEGRADOR 4

Propósito de esta guía

Esta guía tiene como objetivo acompañarte en el desarrollo técnico del Proyecto Integrador 4.

No reemplaza la consigna oficial ni define una única forma de resolver el proyecto. Su función es orientarte con recomendaciones y buenas prácticas para que puedas organizar tu desarrollo y evitar errores comunes.

El proyecto integra varias tecnologías que probablemente no usaste juntas antes: React con TypeScript, Firebase, una función serverless y AWS SES. La clave no es dominar todo antes de empezar, sino entender qué resuelve cada pieza dentro del sistema.

Qué se espera del proyecto

Se espera que construyas una aplicación web completa y funcional que permita a los usuarios gestionar sus tareas de forma segura y persistente. Más allá de que las funcionalidades estén implementadas, se valorará especialmente:

- Que la arquitectura esté organizada por capas y sea fácil de leer.
- Que el tipado con TypeScript sea consistente y refleje una comprensión real del dominio.
- Que las credenciales nunca estén expuestas en el frontend.
- Que las reglas de Firestore protejan correctamente los datos de cada usuario.
- Que el código evidencie comprensión: los commits, la documentación y el uso de IA deben mostrar que las decisiones fueron tomadas con criterio.

Cómo encarar el desarrollo

El flujo del proyecto

Los nueve hitos no son tareas independientes: cada uno habilita el siguiente.

Entender esa cadena antes de empezar ayuda a tomar mejores decisiones desde el inicio.

Hito 1 — Setup inicial

↓ estructura base, dependencias, variables de entorno

Hito 2 — Configuración de Firebase

↓ conexión a Auth y Firestore verificada

Hito 3 — Autenticación

↓ registro, login, logout y sesión persistente

Hito 4 — Rutas protegidas

↓ acceso controlado según estado de autenticación

Hito 5 — Modelo de datos y seguridad

↓ tipos definidos, reglas de Firestore activas

Hito 6 — CRUD de tareas

↓ operaciones completas con sincronización en tiempo real

Hito 7 — Email con AWS SES

↓ envío seguro desde función serverless

Hito 8 — Testing

↓ pruebas unitarias y de componentes con mocks

Hito 9 — Deploy en Vercel

↓ aplicación en producción completamente funcional

Una decisión tomada en el Hito 1, como la estructura de carpetas o el modelo de tipos, va a impactar en todos los hitos siguientes. Vale la pena leer la consigna completa antes de escribir la primera línea de código.

Cómo organizar el proyecto

La consigna sugiere una estructura de carpetas por capas. Antes de empezar a crear archivos, vale preguntarse: ¿qué responsabilidad tiene cada capa? ¿dónde va

la lógica de negocio? ¿dónde van los tipos? ¿dónde van las llamadas a servicios externos?

Una arquitectura bien pensada desde el inicio hace que agregar funcionalidades sea predecible y que los errores sean más fáciles de encontrar. Una arquitectura improvisada acumula deuda técnica que se hace más costosa a medida que el proyecto crece.

1. Setup inicial

Estructura base, dependencias y entorno seguro

¿Qué tener en cuenta?

El setup inicial no es solo instalar dependencias: es tomar decisiones que van a condicionar todo el proyecto. La más importante en esta etapa tiene que ver con la seguridad: ¿cómo se van a manejar las variables de entorno? ¿Está garantizado que ninguna credencial va a llegar al repositorio?

Otra decisión relevante es la estructura de carpetas. La consigna ofrece una sugerencia. Antes de adoptarla o modificarla, vale entender qué principio la guía: separar responsabilidades para que cada parte del código tenga un lugar claro y predecible.

Una advertencia crítica

El archivo `.env` nunca debe subirse al repositorio. Esto no es una buena práctica opcional: es un requisito de seguridad. Si se sube por accidente, las credenciales quedan expuestas aunque se eliminen en un commit posterior.

Antes de hacer el primer commit, verifica que `.env` esté en `.gitignore`.

¿Cómo lo verifico?

- El proyecto corre localmente sin errores.
- El archivo `.env` existe localmente y `.env.example` existe en el repositorio sin valores reales.

- .env está en .gitignore y no aparece en el historial de commits.
- La estructura de carpetas refleja una separación clara de responsabilidades.

2. Configuración de Firebase

Conexión a Authentication y Firestore

¿Qué tener en cuenta?

Antes de configurar Firebase en el código, conviene entender qué hace cada servicio: Authentication maneja identidades de usuarios, Firestore maneja datos. Son dos servicios distintos que se configuran por separado aunque pertenezcan al mismo proyecto de Firebase.

Una pregunta útil en esta etapa: ¿las credenciales de Firebase están en el código o en variables de entorno? Si están hardcodeadas, hay que corregirlo antes de avanzar. Las credenciales en el código son un problema de seguridad que no se resuelve más adelante.

¿Cómo lo verifico?

- La aplicación se conecta a Firebase sin errores en la consola.
- Las credenciales de Firebase están en variables de entorno, no en el código.
- Se puede distinguir en el código dónde se inicializa Authentication y dónde Firestore.

3. Autenticación

Registro, login, logout y sesión persistente

¿Qué tener en cuenta?

La autenticación no es solo un formulario que llama a una función de Firebase. Implica gestionar el estado de la sesión de forma que persista aunque el usuario recargue la página, y comunicar ese estado al resto de la aplicación.

Antes de implementar, vale investigar cómo funciona `onAuthStateChanged` y el patrón Observer. Entender ese mecanismo hace que el código de autenticación tenga mucho más sentido.

Otro aspecto a considerar: los errores de Firebase llegan como códigos internos. ¿Cómo se traducen esos códigos en mensajes útiles para el usuario? Manejar eso bien es parte de construir una experiencia de usuario cuidada.

✅ ¿Cómo lo verifico?

- Un usuario puede registrarse, iniciar sesión y cerrar sesión.
- La sesión persiste al recargar la página.
- Los errores de autenticación se muestran con mensajes comprensibles, no con códigos internos de Firebase.
- Los formularios validan los datos antes de enviarlos.

4. Rutas protegidas

Acceso controlado según estado de autenticación

¿Qué tener en cuenta?

El componente `ProtectedRoute` tiene una responsabilidad concreta: decidir si el usuario puede ver una página o debe ser redirigido. Para implementarlo bien, conviene entender cómo funciona el prop `children` en React y cómo se puede usar para crear componentes que “envuelven” otras vistas.

Un caso que suele generar problemas: cuando la aplicación carga, hay un momento en que no se sabe todavía si el usuario está autenticado o no. Si no se maneja ese estado de carga, puede haber una redirección prematura que da una mala experiencia de usuario.

También vale probar casos extremos: ¿qué pasa si el usuario copia una URL protegida y la pega directamente? ¿Qué pasa si cierra sesión estando en una ruta protegida?

✅ ¿Cómo lo verifico?

- Las rutas protegidas son inaccesibles sin sesión activa.
- Un usuario no autenticado que intenta acceder a una ruta protegida es redirigido correctamente.
- No hay parpadeos ni redireccionamientos inesperados durante la verificación del estado de autenticación.

- Cerrar sesión desde una ruta protegida redirige al login.

5. Crear la Serverless Function

Tipos TypeScript y reglas de Firestore

¿Qué tener en cuenta?

Este hito tiene dos partes que se refuerzan mutuamente: definir la estructura de una tarea en TypeScript y asegurarse de que esa estructura esté protegida en la base de datos.

Antes de escribir código, conviene definir en papel qué campos tiene una tarea, qué tipo tiene cada campo y cuáles son obligatorios. Esa definición es la que luego se expresa como interface en TypeScript y como regla en Firestore.

Las reglas de seguridad de Firestore son críticas: sin ellas, cualquier usuario autenticado podría leer o modificar las tareas de otro. Nunca se debe avanzar con la base de datos abierta.

¿Cómo lo verifico?

Una verificación importante

¿Puede el usuario A leer las tareas del usuario B? Si la respuesta es sí o no se sabe, las reglas de Firestore no están bien configuradas. Probarlo explícitamente es la única forma de estar seguro.

- La interface de tarea en TypeScript tiene todos los campos necesarios con sus tipos correctos.
- Las reglas de Firestore impiden que un usuario acceda a datos de otro.
- Se probó explícitamente que un usuario no puede ver las tareas de otro usuario.

6. CRUD de Tareas

Operaciones completas con sincronización en tiempo real

¿Qué tener en cuenta?

El CRUD de este proyecto tiene una característica particular: la interfaz debe actualizarse automáticamente cuando los datos cambian, sin necesidad de recargar. Eso se logra con onSnapshot, que funciona como una suscripción a cambios en Firestore.

Antes de implementar, vale investigar cómo funciona onSnapshot y por qué es diferente a una consulta normal. También conviene entender qué es un memory leak en este contexto: si la suscripción no se cancela cuando el componente se desmonta, puede generar errores difíciles de diagnosticar.

Otra pregunta útil: ¿cómo se organiza la lógica del CRUD? ¿Está en los componentes directamente o en un custom hook o servicio separado? La segunda opción hace el código más fácil de testear y mantener.

✓ ¿Cómo lo verifico?

- Se pueden crear, editar, eliminar y marcar como completadas las tareas.
- La interfaz se actualiza sin recargar la página después de cada operación.
- Solo se muestran las tareas del usuario autenticado.
- Los estados de carga y error son visibles para el usuario.
- Las suscripciones a Firestore se cancelan correctamente cuando corresponde.

7. Email con AWS SES

Envío seguro desde función serverless

¿Qué tener en cuenta?

Este hito introduce un concepto fundamental: hay operaciones que no pueden hacerse desde el frontend porque expondrían credenciales sensibles. El envío de email con AWS SES es una de ellas.

Antes de implementar, conviene entender por qué no es posible llamar a AWS SES directamente desde el navegador. La respuesta a esa pregunta define la arquitectura: se necesita una función serverless que actúe como intermediario, reciba la solicitud del frontend y realice la llamada a AWS con credenciales que solo existen en el servidor.

Vale implementar este hito después de tener el CRUD funcionando. Es el componente más desacoplado del proyecto y tiene sus propias consideraciones: AWS SES en modo sandbox solo envía a emails verificados, y probar localmente requiere Vercel CLI.

Una advertencia de seguridad

Las credenciales de AWS no deben estar en el frontend bajo ninguna circunstancia.

Si aparecen en el código del cliente, cualquier persona puede extraerlas inspeccionando el navegador. Eso puede generar costos inesperados en la cuenta de AWS o comprometer la seguridad del sistema.

¿Cómo lo verifico?

- La función serverless envía el email correctamente.
- Las credenciales de AWS no son visibles en ningún archivo del frontend.
- El frontend llama a la función serverless, no a AWS directamente.
- Se manejan los estados de carga, éxito y error en la UI.

8. Testing

Pruebas unitarias y de componentes con mocks.

¿Qué tener en cuenta?

El objetivo del testing no es alcanzar un porcentaje de cobertura: es tener confianza en que las partes más críticas del código funcionan como se espera. La pregunta que guía esta etapa es: ¿qué es lo que más le costaría romper a la aplicación?

Los tests de componentes que usan Firebase requieren mocks: simulaciones de los servicios externos que permiten testear el comportamiento del componente sin hacer llamadas reales. Antes de escribir los tests, conviene entender por qué se usan mocks y cómo funcionan en este contexto.

Un test bien escrito tiene tres partes: define una situación inicial, ejecuta una acción y verifica el resultado esperado. Si un test no sigue esa estructura, probablemente no está probando lo que parece.

✓ ¿Cómo lo verifico?

- Los tests corren sin errores con `npm run test`.
- Los componentes principales tienen tests que prueban su comportamiento, no solo que renderizan.
- Los servicios externos (Firebase) están mockeados: los tests no hacen llamadas reales.
- Hay al menos un test que verifica un caso de error o borde.

9. Deploy en Vercel

Aplicación en producción completamente funcional.

¿Qué tener en cuenta?

El deploy no es el último paso del proyecto: es la validación final. Una aplicación que funciona en local pero no en producción tiene problemas que no se detectaron antes. Por eso conviene desplegar temprano y verificar que las funcionalidades principales funcionan en el entorno real.

Las variables de entorno en Vercel se configuran de forma separada al código. Las del frontend deben empezar con `VITE_` para ser accesibles desde el navegador. Las de las funciones serverless no llevan ese prefijo. Confundir esas convenciones es uno de los errores más frecuentes en el deploy.

Antes de considerar el proyecto terminado, vale probar el flujo completo en producción: registro, login, crear una tarea, editarla, marcarla como completada, enviar el email de resumen y cerrar sesión.

✓ ¿Cómo lo verifico?

- La aplicación está accesible desde una URL pública.
- Autenticación, CRUD y envío de email funcionan en producción.
- No hay errores en la consola del navegador en producción.
- Las variables de entorno están configuradas en Vercel, no en el código.
- La aplicación es usable en dispositivos móviles.

Buenas prácticas recomendadas

Arquitectura y código

- Separar la lógica de negocio de los componentes de UI. Los componentes deberían describir qué se muestra, no cómo se obtienen los datos.
- Usar TypeScript de forma consistente: evitar any salvo que sea inevitable y esté justificado.
- Mantener funciones pequeñas con una responsabilidad clara.
- Limpiar suscripciones en useEffect para evitar memory leaks.

Frontend

- Usar diseño mobile-first.
- Utilizar Flexbox o Grid para el layout.
- Mantener estilos simples y consistentes.

Manejo de API

- Usar async/await.
- Manejar errores de red.
- Mostrar estados de carga.

Git

- Realizar commits pequeños y descriptivos.
- No subir el archivo .env.
- Mantener el repositorio público.

Notas y recomendaciones

- Elige un personaje que sea interesante y que tenga personalidad distintiva (puede ser de películas, series, libros, videojuegos, etc.).
- El system/context prompt es crucial: define cómo habla el personaje, qué sabe, qué limitaciones tiene, y que debe dar respuestas cortas apropiadas para chat.
- Prueba el chat extensivamente: la AI debe mantener contexto de la conversación, responder en el tono del personaje, y manejar diferentes tipos de preguntas.
- No expongas nunca la API key en el código del cliente.
- Testea la aplicación en diferentes tamaños usando DevTools del navegador.
- El historial de conversación se mantiene solo durante la sesión (mientras no se recargue la aplicación).
- Usa console.log para debuggear, pero elimina esos logs o coméntalos antes de desplegar.
- Verifica que el deployment en Vercel funciona correctamente incluyendo las serverless functions.

Errores comunes a evitar

- Conectar la AI antes de probar el chat con datos en memoria.
- No manejar el evento popstate: el back/forward del navegador deja de funcionar.
- No enviar el historial completo en cada llamada a Gemini: el personaje pierde contexto entre mensajes.
- Exponer la API key directamente en el código del frontend.
- No manejar errores de la API.
- No documentar cómo ejecutar el proyecto localmente.

Sobre el uso de IA

Puedes utilizar herramientas de IA como apoyo durante el desarrollo. Debes documentar:

- Qué prompts utilizaste.
- Cómo influyeron en tu implementación.
- Qué decisiones tomaste a partir de las respuestas generadas.

Cierre

Esta guía está pensada para ayudarte a organizar tu desarrollo y tomar buenas decisiones técnicas.

El objetivo final es que puedas demostrar que:

- Sabes construir una Single Page Application.
- Puedes integrar una API de inteligencia artificial de forma segura.
- Entiendes cómo estructurar un proyecto frontend de manera profesional.
- Eres capaz de entregar una aplicación funcional, documentada y desplegada.